

Semantic Segmentation: A Comparative Study on DeepLabV2, BiSeNet, PIDNet

Aurona Gashi
Politecnico di Torino
s322791

Lucia Ghezzi
Politecnico di Torino
s325951

Giacomo Maino
Politecnico di Torino
s338682

Elisabetta Roviera
Politecnico di Torino
s328422

Abstract

Semantic segmentation (SS) is a core computer vision task, essential for applications such as autonomous driving and medical imaging, which assigns a class label to each pixel in an image. In this work, we address SS using three deep CNN architectures: DeepLabV2, BiSeNet, and PIDNet, known for their accuracy and efficiency. We rely on two widely adopted and complementary datasets: Cityscapes, composed of real-world urban scenes, and GTA5, a large-scale synthetic dataset. Our evaluation follows a two-stage pipeline: first, we train and validate all models on Cityscapes to establish an in-domain benchmark; then, we assess cross-domain performance by training BiSeNet and PIDNet on GTA5 and testing them on Cityscapes, under a domain-shift and adaptation setup. To mitigate the domain gap, we apply (i) extensive data augmentation on the source domain and (ii) adversarial training, where a domain discriminator promotes the learning of domain-invariant features. We compare the models in terms of segmentation quality—measured by mean Intersection-over-Union (mIoU)—and computational efficiency, including inference latency and floating-point operations (FLOPs). In-domain training on Cityscapes yields up to 56.3% mIoU for DeepLabV2, 51.6% for BiSeNet, and 56.5% for PIDNet. In the cross-domain setting (GTA5 to Cityscapes), PIDNet with data augmentation achieves 31.8% mIoU, while BiSeNet with adversarial training reaches 33.1%. The full implementation and results are available at: <https://github.com/elisabettaroviera/MLDL-Project>

1. Introduction

SS assigns class labels to each image pixel and is tackled using various deep neural networks. Some models prioritize accuracy with higher computational cost, like DeepLabV2 [1], while real-time architectures such as BiSeNet [2] and PIDNet [3] trade some accuracy for faster training and inference.

A key challenge in SS is the need for large, high-quality pixel-level annotations, which are costly and time-consuming. Synthetic datasets provide a scalable alternative with automatic labels but introduce a domain gap: models trained on synthetic data often suffer performance degradation when applied to real-world images [4]. To address this, domain adaptation methods include source data augmentation to increase diversity and visual similarity to the target domain, and adversarial training, where a discriminator encourages domain-invariant feature learning [5].

In this work, we evaluate the SS performance of the aforementioned networks—DeepLabV2, BiSeNet, and PIDNet. First, we conduct in-domain training and evaluation using the Cityscapes dataset for all three models. Then, we perform cross-domain training and evaluation using GTA5 (synthetic) as the training domain and Cityscapes (real) as the testing domain, focusing on BiSeNet and PIDNet. To improve performance in the domain shift setting, we apply both data augmentation (to BiSeNet and PIDNet) and adversarial training strategies (only to BiSeNet).

2. Background and Related Work

Semantic Segmentation. SS is the task of assigning each pixel in an image to a predefined category, enabling detailed scene understanding. It plays a key role in applications like autonomous driving, medical imaging, and robotics. Advances in deep learning, especially convolutional neural networks (CNNs), have greatly improved segmentation accuracy by capturing hierarchical and contextual features [6].

Domain Shift Problem. A key challenge in SS is domain shift: models trained on synthetic datasets often perform poorly on real-world data due to differences in appearance, lighting, and texture. While synthetic data provides abundant easily labeled samples, the gap between synthetic and real distributions hinders generalization. To reduce this gap, data augmentation is used to make source data more visually similar to the target domain [4].

Adversarial Training for Domain Adaptation. Adversarial training is a popular technique to reduce the domain gap in SS. A domain discriminator is trained to distinguish be-

tween source (synthetic) and target (real) outputs, while the segmentation network learns to produce domain-invariant features that fool the discriminator. This alignment of feature distributions improves generalization to real-world data, even when training is primarily done on synthetic datasets [5].

3. Methodology

In this section, we describe the experimental setup adopted in our study, including the datasets, network architectures, loss functions, training strategies, and evaluation metrics.

3.1. Datasets

We briefly describe the two widely used datasets employed to train our models in their reduced versions. The objective is to perform SS on real-world urban scenes from the Cityscapes dataset.

- **Cityscapes** is a large-scale benchmark for urban scene understanding, composed of 2072 finely annotated, high-resolution images (1 572 for training and 500 for validation) at 2048×1024 pixels. It includes 19 semantic classes (e.g., building, road, person, car), which makes it ideal for evaluating real-world segmentation performance [7].

- **GTA5** is a synthetic segmentation dataset derived from the video game Grand Theft Auto V, containing 2 500 high-resolution frames (1914×1052) with pixel-level annotations across 33 classes. Of these, 19 classes are mapped to the Cityscapes labels, making GTA5 a popular source dataset for synthetic-to-real domain adaptation [8].

3.2. Networks

DeepLabV2 [1] uses atrous (dilated) convolutions to expand the receptive field while preserving spatial resolution. Its Atrous Spatial Pyramid Pooling (ASPP) module applies parallel atrous convolutions with varying dilation rates (e.g., 6, 12, 18, 24) to capture multi-scale contextual information, enabling dense feature maps without downsampling. Atrous convolution expands the receptive field by inserting zeros between kernel elements, allowing detailed spatial information to be retained without reducing feature map resolution.

BiSeNet [2] deals with the trade-off between preserving spatial details and obtaining a large receptive field for real-time SS. It consists of three main components.

- **Spatial Path**. It maintains high-resolution spatial information by employing convolutions with small stride, thus preserving fine spatial details.

- **Context Path**. It utilizes a fast downsampling strategy to rapidly enlarge the receptive field and capture rich contextual information.

- **Feature Fusion Module**. It combines the high-resolution

features from the Spatial Path with the context-rich features from the Context Path to generate the segmentation outputs. **PIDNet** [3] is a real-time SS architecture designed to overcome the overshoot problem in two-branch networks, where fine details can be dominated by contextual features. Inspired by PID controllers, PIDNet introduces a three-branch structure with complementary functions.

- **Proportional (P) branch**. It is designed to preserve fine-grained spatial details using high-resolution features.

- **Integral (I) branch**. It captures global and local context to model long-range dependencies.

- **Derivative (D) branch**. It extracts high-frequency boundary features to refine predictions.

The three options available are PIDNet-S, PIDNet-M and PIDNet-L, each of which offers a different trade-off between speed and accuracy.

3.3. Segmentation Losses

The choice of the loss function plays a critical role in the training of SS models, as it directly influences how the network learns to assign pixel-wise labels [9]. Below, we describe the loss functions explored in our experiments.

- **Cross-Entropy Loss (CE)** is a standard pixel-wise loss for SS, defined for multi-class classification as:

$$CE(\mathbf{p}, \mathbf{G}) = - \sum_{i=1}^C G_i \log(p_i)$$

where $\mathbf{G}$ is the one-hot ground truth and $\mathbf{p}$ the predicted probability vector.

- **Lovász-Softmax Loss (L)** is designed to directly optimize the mIoU metric. Although its formulation involves sorting pixel errors and is more complex than typical losses, it has demonstrated superior performance in segmentation tasks where mIoU is the main evaluation metric.

- **Dice Loss (D)** is based on the Dice Similarity Coefficient, this loss addresses class imbalance by maximizing overlap between prediction $\mathbf{P}$ and ground truth $\mathbf{G}$:

$$Dice(\mathbf{P}, \mathbf{G}) = 1 - \frac{2|\mathbf{P} \cap \mathbf{G}|}{|\mathbf{P}| + |\mathbf{G}|}$$

- **Tversky Loss (TV)** is a generalization of Dice Loss with parameters α and β to weight false positives $|\mathbf{P} - \mathbf{G}|$ and negatives $|\mathbf{G} - \mathbf{P}|$:

$$Tversky(\mathbf{P}, \mathbf{G}) = \frac{|\mathbf{P} \cap \mathbf{G}|}{|\mathbf{P} \cap \mathbf{G}| + \alpha|\mathbf{P} - \mathbf{G}| + \beta|\mathbf{G} - \mathbf{P}|}$$

- **Focal Loss (F)** is designed to address class imbalance by down-weighting easy examples and focusing on hard, misclassified pixels:

$$FL(p) = -\alpha_c(1-p)^\gamma \log(p)$$

where p is the ground truth probability, α_c balances class weights, and γ controls focus on hard samples.

3.4. Adversarial losses and weights

To reduce domain shift, we apply adversarial training: a domain discriminator distinguishes source from target features, while the segmentation network learns domain-invariant representations. We describe the adversarial loss functions and weighting strategies used during training.

– **Binary Cross-Entropy Loss (BCE)** is defined as:

$$CE(p, G) = -G \log(p) - (1 - G) \log(1 - p)$$

where G is the ground truth (0 or 1) and p is the predicted probability of the positive class.

– **Hinge Loss (H)** encourages a margin between positive and negative predictions. It’s particularly effective for separating classes and it is defined as:

$$Hinge(P, G) = \max(0, 1 - GP)$$

where P is the prediction and G is the ground truth label (−1 or 1).

– **Mean-Squared-Error Loss (MSE)** also known as L^2 loss, measures the squared difference between predicted values and targets:

$$MSE(P, G) = \frac{1}{2}(P - G)^2$$

The total loss combines segmentation loss $\mathcal{L}_{seg}$ and adversarial loss $\mathcal{L}_{adv}$, scaled by λ_{adv} :

$$\mathcal{L}(s, t) = \mathcal{L}_{seg}(s) + \lambda_{adv} \mathcal{L}_{adv}(t)$$

We consider two weighting strategies:

– **Fixed strategy**: λ_{adv} is set to a constant value throughout training, providing a stable and consistent weighting of the auxiliary loss term.

– **Ramp-up strategy**: λ_{adv} starts from a small initial value and increases linearly up to a predefined maximum over a fixed portion of the training schedule. Denoting the current iteration (batch) t and the maximum number of ramp-up iterations t_{rp} , in the ramp-up strategy the weight is computed as:

$$\lambda_{adv}(t) = \min \left(\lambda_{max}, \lambda_{init} \frac{t}{t_{rp}} \right)$$

This allows the model to prioritize segmentation early on, gradually incorporating the adversarial signal.

3.5. Data augmentation

To narrow the visual domain gap in SS, we apply a structured augmentation strategy to the source dataset (GTA5), aiming to realistically simulate the target dataset (Cityscapes) while preserving semantic consistency. The augmentations used are selected from the *Albumentations* library and categorized into three transformation types:

– **Color augmentations** simulate variations in lighting and camera settings by modifying image color properties. This includes `HueSaturationValue`, `CLAHE`, `GaussNoise`, `RGBShift`, and `RandomBrightnessContrast`.

– **Weather augmentations** replicate environmental conditions commonly found in real-world scenes. This includes `RandomShadow`, `RandomRain`, `RandomFog`, `ISONoise`, and `GaussianBlur`.

– **Geometric augmentations** introduce variability in spatial structure and perspective to improve robustness to viewpoint changes. This includes `RandomCrop`, `Affine`, and `Perspective`.

In order to devise the most useful augmentations, we follow a systematic two-step approach.

1. Category-wise optimization. We first test various combinations of 1, 2, 3, and 4 augmentations within each category using BiSeNet as our base model. This allows us to identify the most effective augmentation subsets per transformation type in terms of improving cross-domain generalization.

2. Cross-category combination. After determining the best-performing combinations within each category, we explore strategies for mixing them across categories. We aim to introduce greater diversity while retaining the strengths of each individual category.

3.6. Evaluation Metrics

When evaluating SS models, it is important to assess prediction quality and computational performance. Below are defined the three key metrics that we used.

– **Mean Intersection over Union (mIoU)** measures the average pixel-wise overlap between the predicted segmentation and ground truth for each class. For class c we can define the **Intersection over Union (IoU)** as:

$$IoU_c = \frac{TP_c}{TP_c + FP_c + FN_c}$$

where TP , FP , and FN denote true positives, false positives, and false negatives, respectively. The mean over all N classes defines the mIoU:

$$mIoU = \frac{1}{N} \sum_{c=1}^N IoU_c$$

– **Latency and Frames Per Second (FPS)** Latency is the average time, in milliseconds (ms), required to process one image, computed over 1000 iterations. FPS is the number of images processed per second, defined as the inverse of the latency. These metrics evaluate runtime efficiency, which is critical for real-time applications where a balance between speed and accuracy is needed.

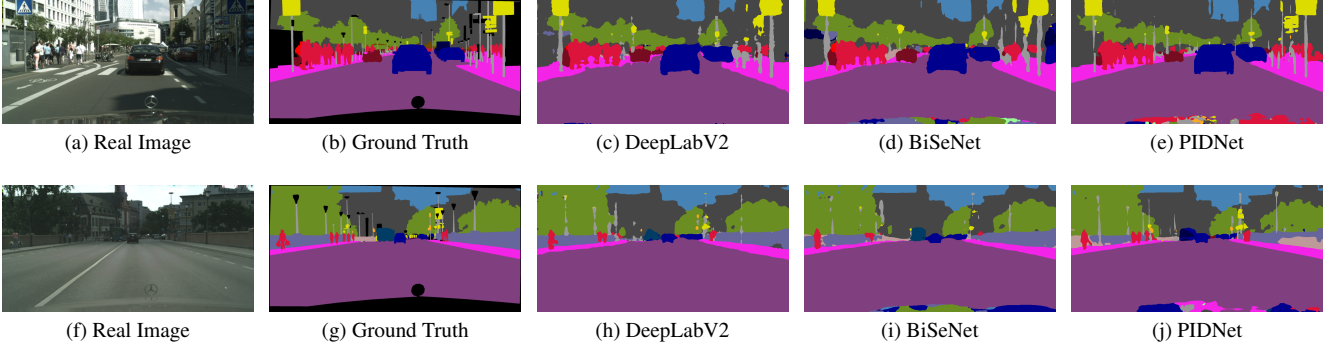


Figure 1. Qualitative comparison of segmentation results on two Cityscapes validation images. Each row displays, from left to right: the input image, ground truth, and predictions (at resolution 1024×512) generated by DeepLabV2, BiSeNet, and PIDNet.

– **Floating Point Operations (FLOPs)** quantify the computational cost by summing all floating-point operations during a forward pass:

$$\text{FLOPs} \approx \sum_{\text{layers}} (\text{operations} \times \text{activation size})$$

FLOPs serve as a hardware-independent measure of complexity and efficiency thus helping compare model efficiency and scalability.

3.7. Training details

All models are trained for 50 epochs—except PIDNet, which is trained for 20—with a fixed input resolution of 1024×512 for both training and evaluation on the Cityscapes dataset, and 1280×720 for GTA5. We adopt ResNet-101 with an ASPP classification module for DeepLabV2, and ResNet-18 for BiSeNet with dual-path structure. All backbones and PIDNet are pre-trained on ImageNet. The optimizer used for all models is Stochastic Gradient Descent (SGD) with a fixed momentum of 0.9. For DeepLabV2 and PIDNet, the weight decay is set to 5×10^{-4} , while for BiSeNet it is set to 10^{-4} . The batch size is 3 for DeepLabV2 and 4 for BiSeNet and PIDNet. The learning rate (LR), whose value is initially set to LR_0 , follows a polynomial decay schedule, defined as $LR(t) = LR_0 \cdot (1 - t/t_{\max})^p$, where t is the current iteration (batch), $t_{\max}$ the maximum number of iterations (epochs*number of batches), and p the power. All other hyperparameters, such as LR, are individually tuned and reported separately.

4. In-domain experiments and results

In this section, we evaluate model performances in an in-domain setting, where both the training and test data are drawn from the Cityscapes dataset.

DeepLabV2. We perform a systematic study by combining the standard pixel-wise CE loss with L, D, and F losses

in convex mixtures. In parallel, we sweep the initial LR in the range $[10^{-1}, 10^{-4}]$. To improve fine-tuning in the final training stages, we freeze layers 1, 2, 3, 4 and the first Batch Normalization layer of the ResNet backbone during the last 5 epochs. This strategy, adopted in some of our experiments, aims to concentrate learning on the deeper layers and better refine task-specific features.

To stabilize early training, following the proposal in [10], we apply a *linear warm-up phase* during the first epochs, gradually increasing the learning rate from a small initial value (i.e., 10^{-6}) up to the target initial LR. This warm-up helps prevent unstable weight updates at the beginning of training. After this initial stage, we switch to a standard polynomial decay schedule (power = 0.9).

To further address class imbalance, we employ a *weighted loss strategy*, where classes are penalized differently based on their performance. Specifically, we compute class weights from the inverse of the per-class mIoU measured on the training set at the previous epoch. Letting mIoU_c denote the mIoU of class c , we define the weight w_c as:

$$w_c = \frac{1/(\text{mIoU}_c + 10^{-6})}{\sum_j 1/(\text{mIoU}_j + 10^{-6})}.$$

These normalized weights are applied to the CE term of the loss to emphasize under-performing classes. Other loss components remain unweighted.

The **best-performing configuration** combines CE and F losses in equal proportion (0.5 each), with the CE term re-weighted as previously described. The warm-up phase spans the first 3 epochs. To further refine optimization, the exponent γ in the F loss is dynamically adjusted: it starts at 2, is set to 3 from epoch 31, and increased to 4 at epoch 43. The highest mIoU is achieved precisely at epoch 46, reaching a peak value of **56.30%**.

BiSeNet. An initial screening on a 25% subset is performed to evaluate convex combinations of CE, L, TV,

Model	Loss	LR	mIoU (%)		Lat	FLOPs	Param
			Fin	Max			
DL	0.7CE+0.3L	0.0002	53.79	53.98	238.7	374.7	2.8M
DL	0.5CE+0.5F	0.0003	53.99	53.99	249.0	374.7	2.8M
DL	0.5CE+0.5F	0.0005	56.26	56.26	273.5	374.7	43.8M
DL	0.5CE*+0.5F	0.0005	55.65	56.30	277.6	375.5	43.8M
BN	1.0CE	0.0063	49.97	50.67	17.5	25.8	12.6M
BN	0.6CE+0.2L+0.2TV	0.0063	50.96	51.59	15.8	25.8	12.6M
BN	0.7CE+0.3TV	0.0063	51.04	51.04	17.0	25.8	12.6M
PN_S	1.0CE	0.0100	51.39	51.89	10.5	12.7	7.7M
PN_M	1.0CE	0.0100	56.49	56.49	27.2	46.3	28.8M
PN_L	1.0CE	0.0100	53.95	54.51	36.5	72.0	37.3M

Table 1. Comparison of DeepLabV2 (DL), BiSeNet (BN), and PIDNet (PN) variants. * denotes class-weighted loss. Latency (**Lat**) in ms; **FLOPs** in GFLOPs; Parameters (**Param**) in millions (M). **Fin** denotes the final mIoU; **Max** the peak mIoU reached.

$\alpha = \beta = 0.5$ (see Section 3.3), and D losses, while sweeping the LR across $[10^{-1}, 10^{-4}]$. The most promising configurations are then selected for retraining on the complete dataset. To determine a suitable LR, we apply two complementary strategies. First, we follow the linear LR scaling rule proposed in [10], which states that the LR should scale linearly with the batch size with LR_{base} and BS_{base} as in [2]:

$$\text{LR}_{\text{new}} = \text{LR}_{\text{base}} \times \frac{\text{BS}_{\text{new}}}{\text{BS}_{\text{base}}} \Rightarrow \text{LR} = 0.025 \times \frac{4}{16} = 0.00625.$$

Second, we validate this value through a one-pass LR range test. Starting from $\text{LR}_0 = 10^{-6}$, the LR is increased up to $\text{LR}_{\text{end}} = 10^{-1}$ using an exponential schedule: $\text{LR}(t) = \text{LR}_0 \times \left(\frac{\text{LR}_{\text{end}}}{\text{LR}_0}\right)^{t/t_{\text{end}}}$, where t denotes the current iteration and $t_{\text{end}} = 1000$ is the total number of test iterations. The loss is computed as the sum of BiSeNet’s main and auxiliary outputs, and smoothed using an exponential moving average. This procedure reveals a broad low-loss plateau centered around 6×10^{-3} , confirming that the selected LR of 0.00625 lies well within a stable training region.

We also conduct a comparative analysis of LR decay (power = 0.9) applied per epoch versus per batch. Results indicate that batch-wise decay yields slightly higher performance peaks and training stability over time. Consequently, all experiments adopt LR decay at every batch.

Two configurations emerge as the most promising candidates. The first combines CE (0.6), L (0.2), and TV (0.2), while the second adopts a simpler CE (0.7) + TV (0.3) scheme. Although the CE+L+TV setup reaches a slightly higher peak performance at intermediate epochs (notably at epoch 40), we select CE+TV as the **best-performing configuration** and use it as the reference model in all subsequent cross-domain studies. This choice is motivated by its higher final mIoU of **51.04%**, reduced training variance, and slightly improved inference speed.

PIDNet. We experiment with its three variants: **S**, **M**, and **L**, corresponding respectively to the smallest (lightweight)

and largest (heavier) architectures. The total loss function used is composed of multiple terms, each weighted by a scalar hyperparameter: $\mathcal{L} = \lambda_0 \mathcal{L}_0 + \lambda_1 \mathcal{L}_1 + \lambda_2 \mathcal{L}_2 + \lambda_3 \mathcal{L}_3$, where $\mathcal{L}_0$ is the auxiliary segmentation loss (a CE applied on the intermediate semantic head), $\mathcal{L}_1$ the weighted boundary BCE, $\mathcal{L}_2$ the CE, and $\mathcal{L}_3$ the boundary-aware CE, defined as:

$$\mathcal{L}_3 = - \sum_{i,c} \mathbf{1}_{\{b_i > T\}} s_{i,c} \log \hat{s}_{i,c}$$

where T is a predefined threshold, and b_i , $s_{i,c}$, and $\hat{s}_{i,c}$ denote the boundary head output, segmentation ground truth, and prediction for the i -th pixel and class c , respectively.

We first adopt the same setting as provided in [3]: $\lambda_0 = 0.4$, $\lambda_1 = 20$, $\lambda_2 = 1$, $\lambda_3 = 1$, $T = 0.8$. To improve training, we conduct numerous experimental runs in which we modify both the LR and the weighting of $\mathcal{L}_1$ and try the dropout strategy. We choose to perform training and evaluation for 20 epochs, as further training does not yield significant improvements. The **best-performing configuration** is obtained without dropout, by adopting a LR of 0.01 with a polynomial decay (power = 0.95), and by applying exponential decay to λ_1 : $\lambda_1 = 20 \cdot (0.9)^{\frac{\text{epoch}}{10}}$. We ultimately select **PIDNet-M** as the reference model for all subsequent cross-domain studies, as it provides a favorable trade-off between segmentation accuracy and inference speed. It achieves the highest mIoU of **56.49%**.

4.1. Comparison and analysis of results

The results in Table 1 reveal clear trade-offs between accuracy, latency, and model complexity. DeepLabV2 achieves high peak mIoU (56.30%) using class-weighted CE+F loss, but only in its full configuration with all layers unfrozen, which results in significantly increased latency (277.6ms) and parameter count (43.8M). Earlier runs with frozen backbone blocks reduce complexity (2.8M) but yield lower performance. BiSeNet offers real-time inference with latency below 20ms and moderate size (12.6M), reaching 51.04% mIoU with CE+TV; its improvement over the CE-only baseline (+1.07%) highlights the effectiveness of boundary-aware regularization in compact models. PIDNet variants explore different speed–accuracy trade-offs: the Small model is the fastest (10.5ms, 7.7M) with modest accuracy, the Large model improves mIoU by just +2.56% at much higher cost, while the Medium variant strikes the best balance—achieving the highest mIoU (56.49%) with 27.2ms latency.

Figure 1 provides a qualitative perspective on these quantitative trends. DeepLabV2 reliably fills void regions by extending nearby class predictions, resulting in clean masks and crisp class boundaries, though very thin structures may be slightly blurred. BiSeNet produces smooth, artifact-free masks with consistent boundaries, but can oversmooth and occasionally introduce noise at class edges. PIDNet yields

Model	Type	mIoU	road	sidewalk	building	wall	fence	pole	traffic light	traffic sign	vegetation	terrain	sky	person	rider	car	truck	bus	train	motorcycle	bicycle
BiSeNet	None	17.85	22.92	5.26	56.30	3.52	2.85	5.21	7.54	2.34	62.11	6.12	48.17	29.09	1.09	31.78	0.54	0.81	0.00	0.00	0.00
PIDNet	None	20.15	43.18	11.30	66.23	4.04	7.18	18.68	17.16	9.68	68.12	3.92	62.78	29.68	4.80	29.10	2.61	1.71	0.14	2.33	0.17
BiSeNet	aug_1	30.65	70.35	20.35	72.00	12.68	11.64	20.05	17.75	13.06	76.39	24.48	74.70	38.13	2.94	74.45	20.22	16.82	3.87	10.07	2.36
BiSeNet	aug_2	29.75	69.39	15.55	70.12	19.91	7.74	21.80	16.22	7.28	76.63	22.14	74.03	38.93	4.72	62.29	16.45	8.00	0.00	3.98	0.24
PIDNet	aug_1	30.71	65.80	28.52	71.87	14.79	15.27	30.28	21.77	9.07	74.87	22.73	77.37	38.98	8.37	44.29	19.91	7.32	22.51	9.17	0.63
PIDNet	aug_2	31.76	70.17	30.64	68.12	18.00	8.87	30.58	18.72	14.07	78.42	28.03	78.75	41.02	9.03	70.98	20.27	9.60	2.19	4.47	1.43

Table 2. Per-class validation mIoU on the Cityscapes dataset for BiSeNet and PIDNet models under different augmentation strategies. The column **Type** indicates the augmentation applied: None is the baseline (no augmentation), while aug_1 and aug_2 refer to the combinations detailed in Section 5.2.

the sharpest object contours and best preserves fine details such as poles and signs, but often mislabels void pixels near the image bottom, assigning them to unrelated classes. These visual differences align with the metrics, highlighting how model architecture impacts both global performance and local segmentation accuracy.

5. Cross-domain experiments and results

In this section, we focus on the cross-domain setting, where models are trained on the synthetic GTA5 dataset and evaluated on the real-world Cityscapes dataset.

5.1. Baseline Training

In this setting, BiSeNet and PIDNet are trained without any domain adaptation, using the best-performing architectures and hyperparameters from Section 4. This serves as a baseline for evaluating adaptation strategies.

The per-class validation mIoU for these baseline models is shown in the first two rows of Table 2.

BiSeNet achieves a mIoU of **17.85%**. Performance is particularly low on *sidewalk* (5.26%) and *truck* (0.54%), indicating difficulty in generalizing from synthetic to real-world imagery.

PIDNet reaches a higher mIoU of **20.15%**. Notably, PIDNet outperforms BiSeNet on *road* (43.18% vs. 22.92%) and *pole* (18.68% vs. 5.21%), demonstrating a stronger ability to preserve fine structural details under domain shift.

These results establish PIDNet as the more robust baseline in the cross-domain scenario; in the following section, we will apply and compare data augmentation on both models.

5.2. Training with data augmentation

We conduct all augmentation experiments using BiSeNet as the base model, aiming at improving cross-domain performance from GTA5 to Cityscapes and following a two-phase evaluation setup.

1. Selection phase. We train BiSeNet on 25% of the GTA5 dataset for 15 epochs and test on a subset of Cityscapes training set (approximately one third of the full set) to evaluate different augmentation strategies, based on mIoU. Augmentations are applied with a probability of 50% and we

Augmentation configurations	mIoU (%)			epochs > 25%
	Fin	Max	Mean	
2 Random Color	27.46	28.68	25.01	27
aug_1 = 2 Random Color or com_w	26.70	29.75	26.12	36
Only com_c1	26.54	28.20	24.72	23
Only com_c2	22.21	27.06	23.64	15
Only com_c3	22.55	25.56	21.90	1
Only com_c4	25.02	28.66	25.47	33
One of com_c1, com_c2, com_c3, com_c4	25.14	31.16	25.73	37
One of com_c1, com_c2, com_w	26.38	27.17	24.83	27
aug_2 = One of com_c1, com_c2, com_c3, com_w	28.48	30.65	26.39	41

Table 3. Summary of mIoU metrics across different augmentation configurations. epochs > 25% is the number of epochs with mIoU greater than 25. Where:

com_w = {RandomFog, RandomRain, ISONoise},

com_c1 = {HueSaturationValue, RGBShift, RandomBrightnessContrast},

com_c2 = {RGBShift, RandomBrightnessContrast},

com_c3 = {HueSaturationValue, RGBShift, CLAHE},

com_c4 = {GaussNoise, RGBShift, RandomBrightnessContrast}

also retain the original image. Following the strategy explained in Section 3.5, we find the most effective strategies.

2. Final training phase. The best augmentation strategies from the selection phase are used to train BiSeNet on the full GTA5 dataset for 50 epochs. Evaluation is conducted on the full Cityscapes test set. Table 3 presents a comparison of the results of the final fully trained models. The selected **best-performing augmentation** strategies are:

– **aug_1** = "Apply 2 random Color transformations together or com_w, each with equal probability", with mean mIoU of 26.12% and best mIoU of **29.75%** at epoch 45.

– **aug_2** = "Apply one of com_c1, com_c2, com_c3, com_w, each with equal probability" with mean mIoU of 26.39% and best mIoU of **30.65%** at epoch 44.

For PIDNet, we skip the selection phase and directly apply *aug_1* and *aug_2* in the final training, achieving a best validation mIoU of **31.76%** with *aug_2*, outperforming the BiSeNet counterpart.

5.3. Adversarial training

We assess the impact of different adversarial loss functions and weighting strategies using BiSeNet as the base model. The evaluation is conducted in two phases.

Type	Loss	Lambda	mIoU	road	sidewalk	building	wall	fence	pole	traffic light	traffic sign	vegetation	terrain	sky	person	rider	car	truck	bus	train	motorcycle	bicycle
None	BCE	0.001	24.98	52.87	19.06	67.58	8.59	8.27	18.57	15.90	9.73	65.18	6.37	65.75	39.07	0.53	38.44	4.74	1.61	0.00	2.42	0.00
aug_1	H	0.002	31.03	69.50	23.53	72.17	20.69	11.46	22.25	16.95	8.98	76.17	20.67	67.21	35.95	5.87	64.07	18.12	16.78	3.81	4.41	0.00
aug_2	H	0.002	33.08	71.54	29.39	72.47	25.92	11.52	20.15	14.86	8.90	76.33	24.13	71.67	35.57	4.24	75.69	22.81	15.52	10.25	4.53	0.00
aug_1	H	Ramp-up	31.14	70.81	24.04	71.48	20.64	7.99	20.05	14.94	9.96	75.53	24.00	68.87	37.15	5.22	68.08	18.79	16.84	3.84	2.36	0.00
aug_2	H	Ramp-up	32.64	71.35	37.83	72.13	23.50	11.55	21.93	16.01	9.72	75.48	25.45	67.92	39.38	3.58	68.03	19.00	10.76	9.17	4.66	0.00

Table 4. Per-class validation mIoU on the Cityscapes dataset for BiSeNet trained with adversarial domain adaptation and the best data augmentation strategies. The column **Loss** specifies the adversarial loss function (BCE or H), and **Lambda** indicates the adversarial weight—fixed values when given numerically, or a ramp-up schedule from 10^{-6} otherwise, and **Type** the augmentation strategy (refer to Section 5.2).

Loss_adv	Lambda_adv	mIoU (%)			epochs > 25%
		Fin	Max	Mean	
BCE	Fixed 0.001	23.67	23.77	22.37	0
BCE	Fixed 0.002	23.72	25.76	23.57	2
H	Fixed 0.001	24.50	26.53	23.93	4
H	Fixed 0.002	26.10	26.10	24.53	8
H	Ramp-Up 10^{-6}	25.33	27.11	24.75	15
H	Ramp-Up 10^{-4}	25.02	26.24	24.55	10
MSE	Ramp-Up 10^{-6}	21.57	24.94	22.37	0
MSE	Ramp-Up 10^{-4}	23.01	24.83	23.04	0

Table 5. BiSeNet mIoU metrics across different adversarial loss (**Loss_adv**) and weight (**Lambda_adv**) configurations. **Mean** is the average mIoU over 50 epochs.

1. Selection phase. We train BiSeNet on the GTA5 dataset for 25 epochs and validate on a subset of the Cityscapes training set. We test different combinations of adversarial losses (BCE, H, MSE) and weighting strategies—fixed $\lambda_{adv} \in \{0.001, 0.002\}$ and ramp-up strategies with $\lambda_{init} \in \{10^{-4}, 10^{-6}\}$ linearly increasing to $\lambda_{max} = 0.001$ over 40% of training iterations. Results are summarized in Table 5.

The **best-performing strategies** consist in using Hinge loss for the discriminator, paired with either a fixed weight of $\lambda_{adv} = 0.002$ (providing as best mIoU **26.1%**) or a ramp-up from $\lambda_{init} = 10^{-6}$ to $\lambda_{max} = 0.001$ (giving as best mIoU **27.11%**).

2. Final training phase. Based on the best-performing strategies from the selection phase, we retrain BiSeNet on the full GTA5 dataset for 50 epochs and evaluate on Cityscapes validation set. In this phase, we also apply the two best augmentation strategies *aug_1* and *aug_2* identified in Section 5.2. As a baseline, we reproduce the setup from [11], using BCE loss with $\lambda_{adv} = 0.001$ and no augmentation. The **best configuration** is obtained with *aug_2*, reaching a maximum mIoU of **33.08%**.

5.4. Comparison and analysis of results

As shown in Table 2, applying data augmentation significantly improves cross-domain performance over the no-augmentation baseline. For BiSeNet, the mIoU increases from 17.85% to 30.65% with strategy *aug_1*, and for PIDNet, from 20.15% to 31.76% using *aug_2*. These results

confirm the effectiveness of domain-targeted augmentations in bridging the synthetic-to-real gap.

Interestingly, PIDNet consistently outperforms BiSeNet across all configurations, suggesting that its architecture is more robust to domain shift when trained with augmented data. Moreover, the improvements are particularly evident for fine-grained classes such as *pole*, *traffic light*, and *person*, which suffer most in the unaugmented baseline.

Adversarial training, when combined with data augmentation, further enhances cross-domain performance, as shown by the results in Table 4. The baseline, using BCE loss with fixed $\lambda_{adv} = 0.001$ and no augmentation, achieves 24.98% mIoU. The best configuration boosts this to 33.08%, a significant improvement.

Among the tested losses, H consistently outperforms both BCE and MSE, confirming prior findings that margin-based adversarial objectives yield more stable domain alignment. Additionally, the ramp-up strategy for λ_{adv} proves effective: it allows the segmentation network to first focus on learning task-relevant features, gradually incorporating the adversarial signal to refine domain invariance. The ramp-up version with *aug_2* yields 32.64% mIoU, nearly matching the fixed-weight setup.

Figure 2 illustrates how BiSeNet’s performance evolves: without augmentation, roads and thin structures like poles are inconsistently segmented and bleed into neighboring classes, while applying targeted color and weather augmentations restores uniform road masks and revives slender details. Adding adversarial training smooths the overall mask and sharpens class transitions at the expense of slight blurring of very fine elements. These visual outcomes reflect the mIoU gains, showing that augmentations restore detail and adversarial loss enforces consistency, highlighting how careful tuning with augmentation reduces the domain gap in SS.

6. Discussion and findings

This study set out to evaluate three contemporary segmentation networks—DeepLabV2, BiSeNet and PIDNet—in both in-domain (Cityscapes) and cross-domain (GTA5 to Cityscapes) settings, isolating the impact of architecture,

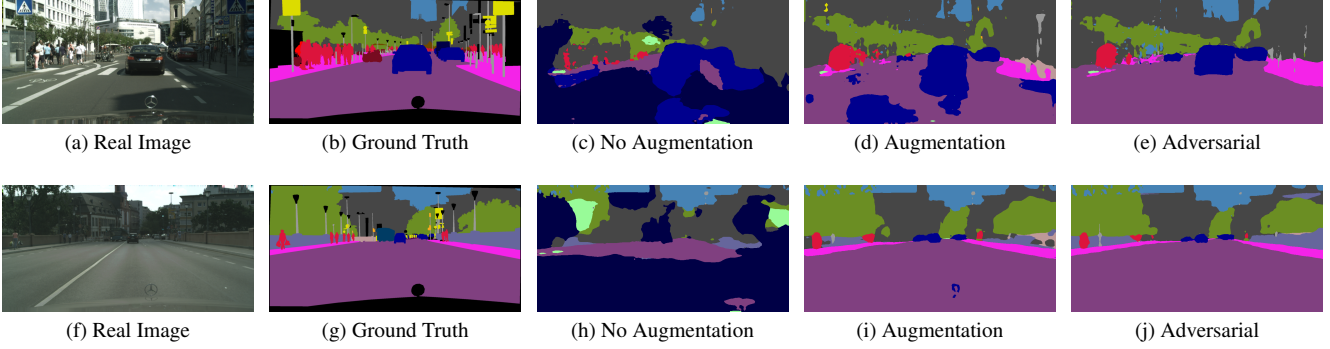


Figure 2. Qualitative comparison of cross-domain adaptation segmentation results on Cityscapes. Each row shows (from left to right): the input image, ground truth, and predictions from BiSeNet with no augmentations, BiSeNet with best augmentation, and with Adversarial.

data augmentation and adversarial adaptation. Our results lead to several key insights.

In-Domain Trade-Offs. PIDNet-M achieves the highest mIoU (56.49%) while being the fastest model, training $7\times$ faster than DeepLabV2 and $2\times$ faster than BiSeNet. DeepLabV2 reaches comparable accuracy (56.30%) but with high latency and large model size. BiSeNet is faster than DeepLabV2 but less accurate (51.59%). Overall, PIDNet-M offers the best trade-off between accuracy and efficiency for real-time applications.

Cross-Domain Augmentation Approach. Introducing targeted color and weather augmentations yields a consistent +10 percentage points boost in cross-domain mIoU for both BiSeNet and PIDNet, reaching 30.65% and 31.76%, respectively. This confirms that simulating Cityscapes visual characteristics substantially narrows the synthetic-to-real gap, particularly for thin or rare classes (e.g., poles, traffic lights, riders), where unaugmented models underperform.

Adversarial Adaptation Benefits. Adding H loss-based adversarial training further improves mIoU by 2-3 percentage points, with BiSeNet reaching 33.08%. The ramp-up schedule for λ_{adv} proves nearly as effective as a fixed weight, suggesting that gradually introducing domain alignment avoids destabilizing early feature learning.

Future Directions. Looking ahead, future research could investigate semi-supervised self-training on unlabeled Cityscapes frames, multi-discriminator adversarial frameworks to model a wider range of style variations, and feature-level alignment techniques (e.g., contrastive domain losses). Additionally, curriculum-based augmentation schedules and adaptive adversarial weighting could further stabilize training and lessen dependence on costly annotations, moving closer to truly robust, real-world segmentation.

References

[1] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, “Deeplab: Semantic image segmentation with

deep convolutional nets, atrous convolution, and fully connected crfs,” in *IEEE TPAMI*, vol. 40, pp. 834–848, 2017. 1, 2

[2] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, “Bisenet: Bilateral segmentation network for real-time semantic segmentation,” in *ECCV*, pp. 325–341, 2018. 1, 2, 5

[3] J. Xu, Z. Xiong, and S. P. Bhattacharyya, “Pidnet: A real-time semantic segmentation network inspired by pid controllers,” *arXiv preprint arXiv:2206.02066*, 2022. 1, 2, 5

[4] S. Zhao, X. Yue, S. Zhang, B. Li, H. Zhao, B. Wu, R. Krishna, J. E. Gonzalez, A. L. Sangiovanni-Vincentelli, S. A. Seshia, and K. Keutzer, “A review of single-source deep unsupervised visual domain adaptation,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 473–493, 2022. 1

[5] Y.-H. Tsai, W.-C. Hung, S. Schulter, K. Sohn, M.-H. Yang, and M. Chandraker, “Learning to adapt structured output space for semantic segmentation,” in *CVPR*, pp. 7472–7481, 2018. 1, 2

[6] S. Hao, Y. Zhou, and Y. Guo, “A brief survey on semantic segmentation with deep learning,” *arXiv preprint arXiv:2007.04047*, 2020. 1

[7] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, “The cityscapes dataset for semantic urban scene understanding,” in *CVPR*, pp. 3213–3223, 2016. 2

[8] S. R. Richter, V. Vineet, S. Roth, and V. Koltun, “Playing for data: Ground truth from computer games,” in *ECCV*, pp. 102–118, 2016. 2

[9] S. Jadon, “A survey of loss functions for semantic segmentation,” *arXiv preprint arXiv:2006.14822*, 2020. 2

[10] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, “Accurate, large minibatch sgd: Training imagenet in 1 hour,” *arXiv preprint arXiv:1706.02677*, 2017. 4, 5

[11] W. Tranheden, V. Olsson, J. Pinto, and L. Svensson, “Dacs: Domain adaptation via cross-domain mixed sampling,” in *WACV*, pp. 1379–1389, 2021. 7